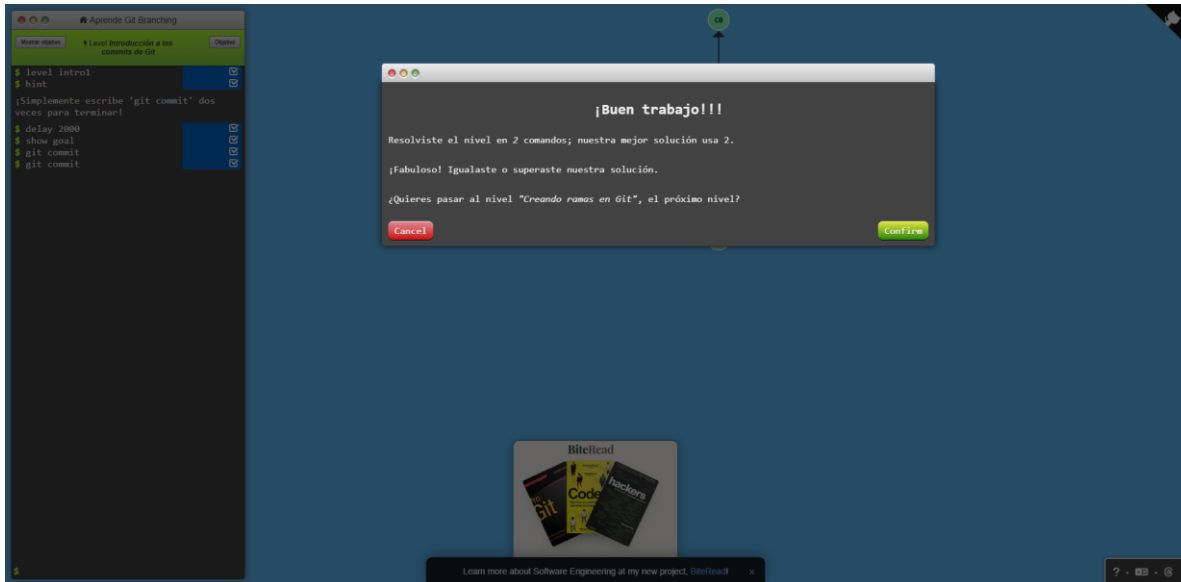


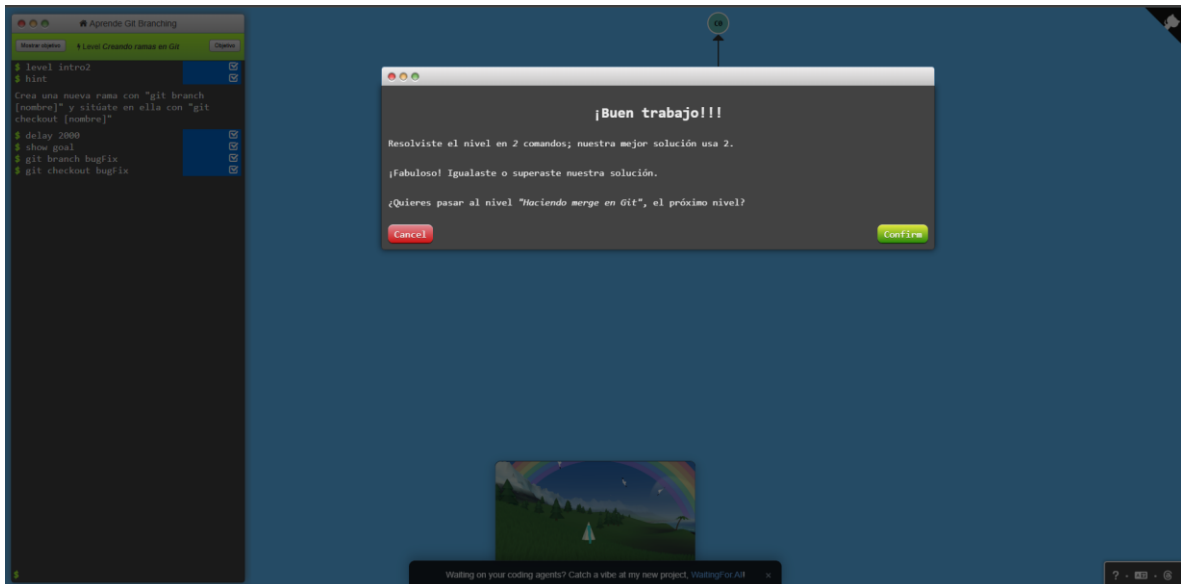
TALLER APRENDE GIT BRANCHING

PUNTO1:



Git Commit: crea un repositorio adicional basado al padre es decir, al momento de sacar el nuevo repositorio apartir del C2 crea el C3

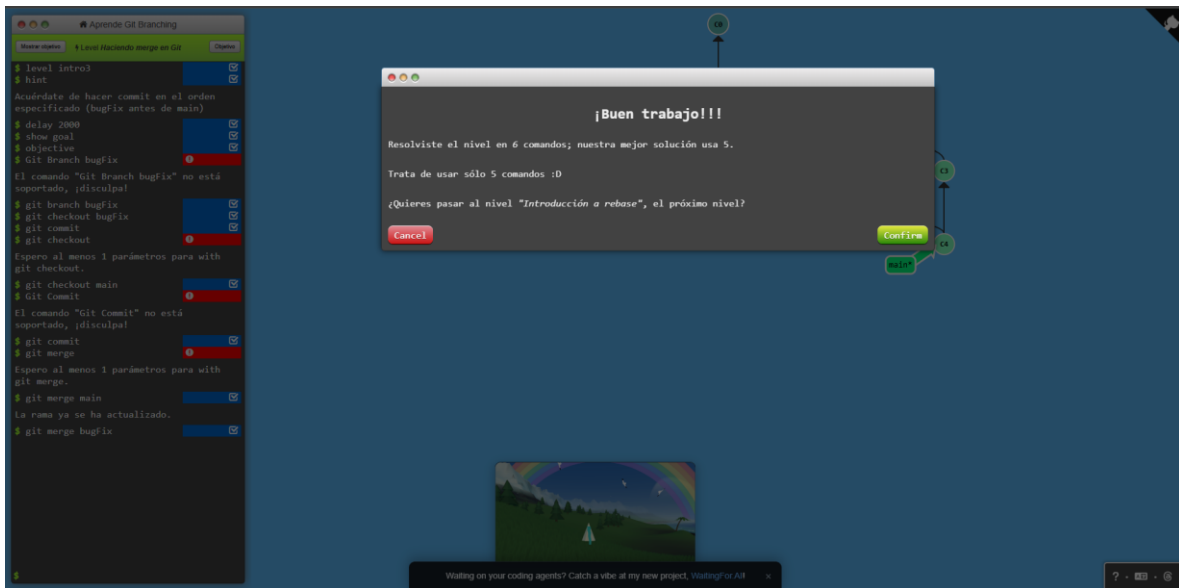
PUNTO2:



Git Branch[Nombre]: Creamos una nueva rama y se le asigna un nombre completamente único hace referencia al commit al cuál se le asignó.

git checkout [name]: Esto va a situarnos en esa rama antes de hacer un commit con nuestros cambios.

PUNTO 3:



Git Merge [Nombre]: Crea una rama completamente diferente aparte de la principal para realizar cambios y luego unificarlas para un resultado mejor.

Merge[Nombre]: Lo que significa que cada rama contiene todo el trabajo que hay en el repositorio los une.

- Crea una nueva rama, llamada bugFix
- Haz checkout de la rama bugFix usando git checkout bugFix
- Crea un commit
- Vuelve a main con git checkout
- Crea otro commit
- Haz merge de la rama bugFix a main usando git merge

git branch bugFix

git checkout bugFix

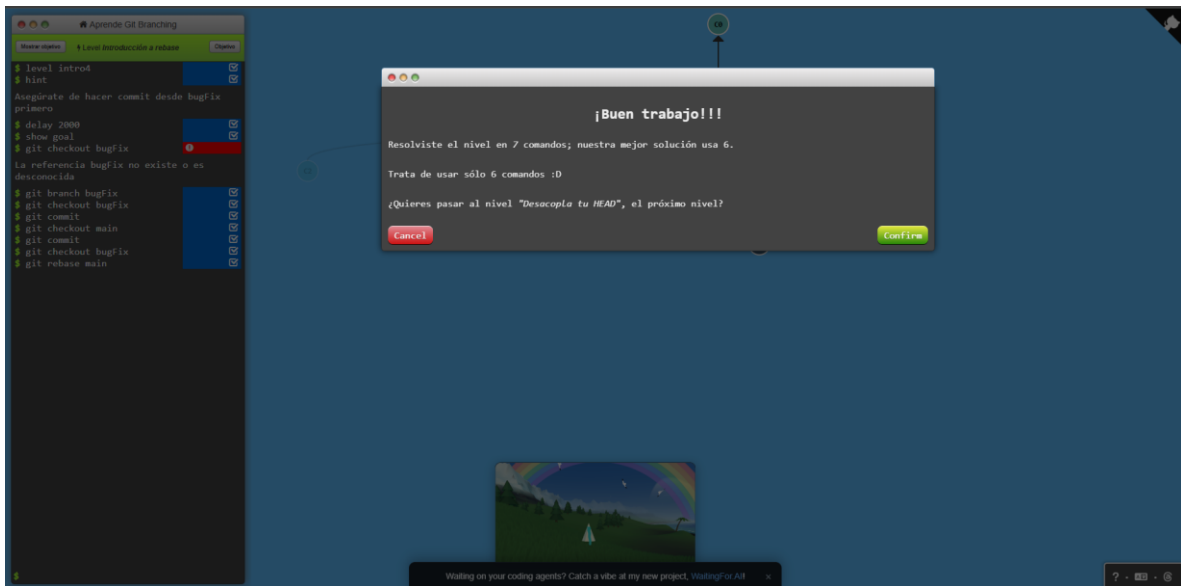
git commit

git checkout

git commit

git merge

PUNTO 4:



Git Rebase: Nos gustaría mover nuestro trabajo de bugFix directamente sobre el trabajo de main. De ese modo, parecería que esas dos tareas se desarrollaron secuencialmente, cuando en realidad se hicieron en paralelo.

- Haz checkout de una nueva rama llamada bugFix
- Crea un commit
- Vuelve a la rama main y crea otro commit
- Haz checkout en bugFix otra vez y haz rebase sobre main

git branch bugFix

git checkout bugFix

git commit

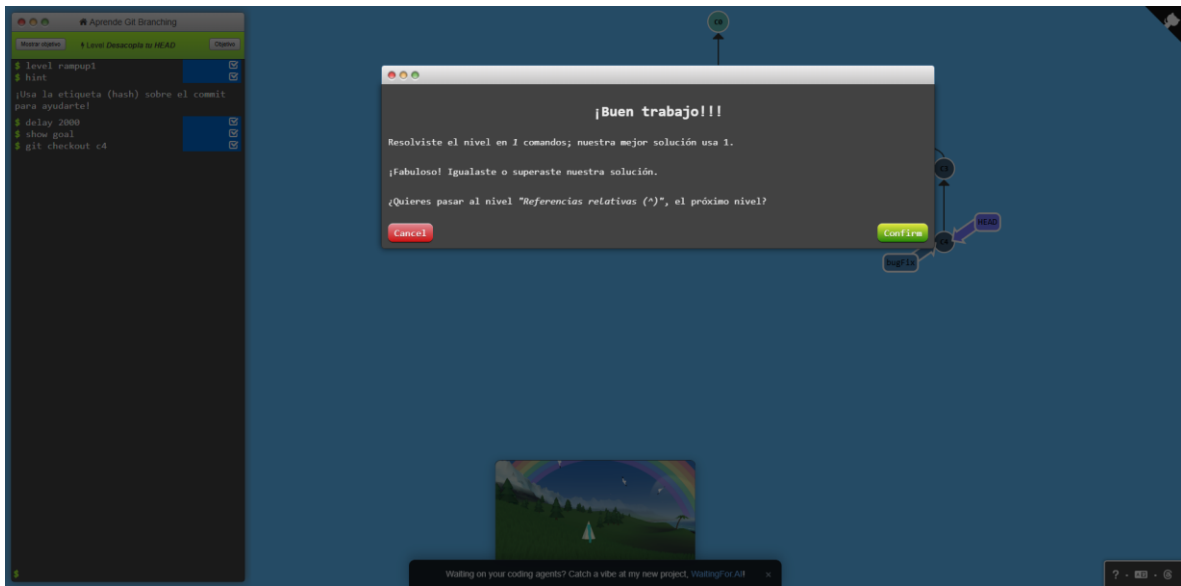
git checkout main

git commit

git checkout bugFix

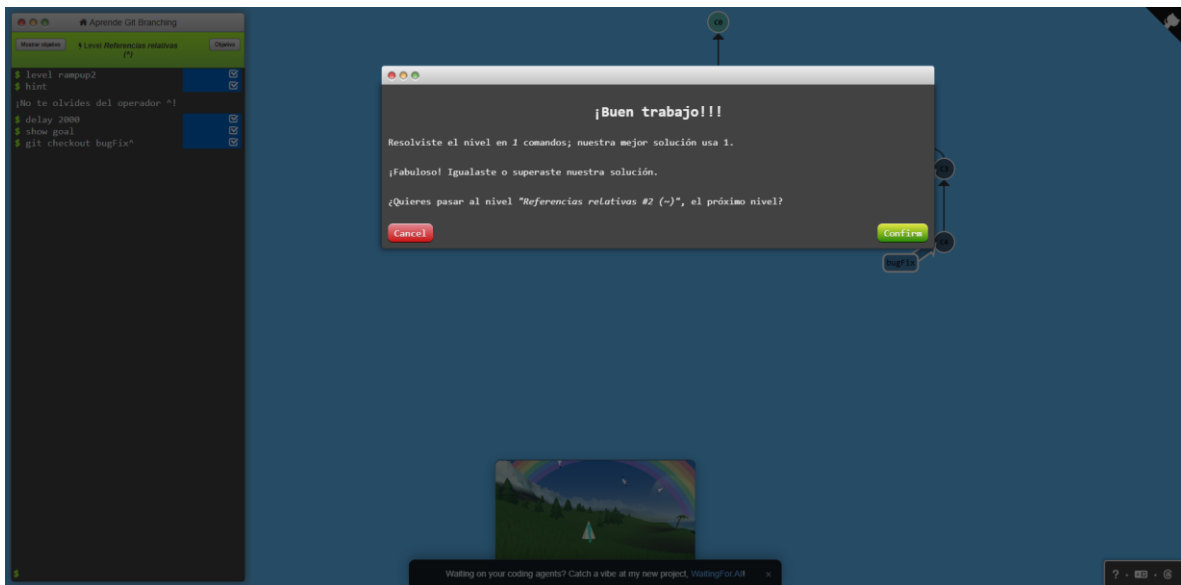
git rebase main

PUNTO 5:



Git Checkout C?: Hacia donde necesitamos q se mueva o se posicione el HEAD ~

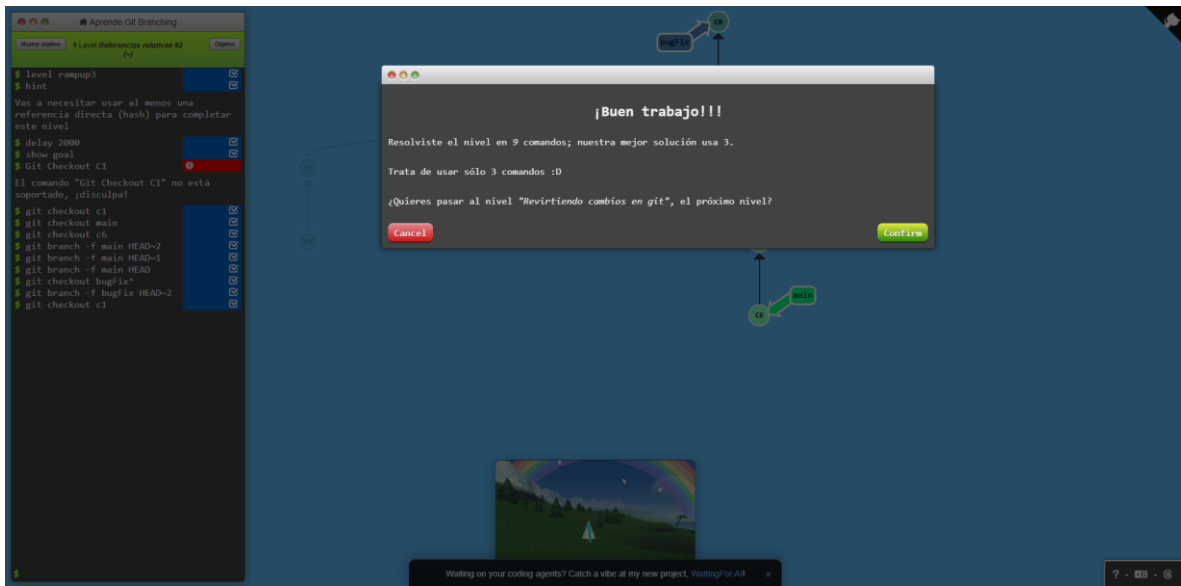
PUNTO 6:



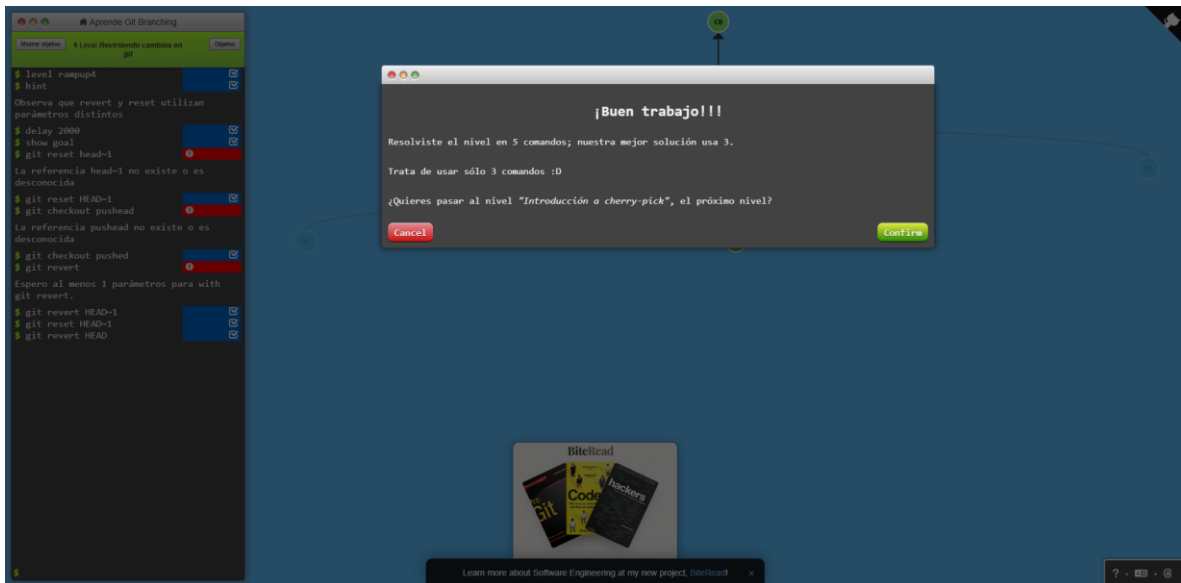
Git Checkout [Nombre] ^: Permite al HEAD atrasarse en 1 por cada ^.

PUNTO 7:

git branch -f main HEAD~3: El main va tres veces más atrás.



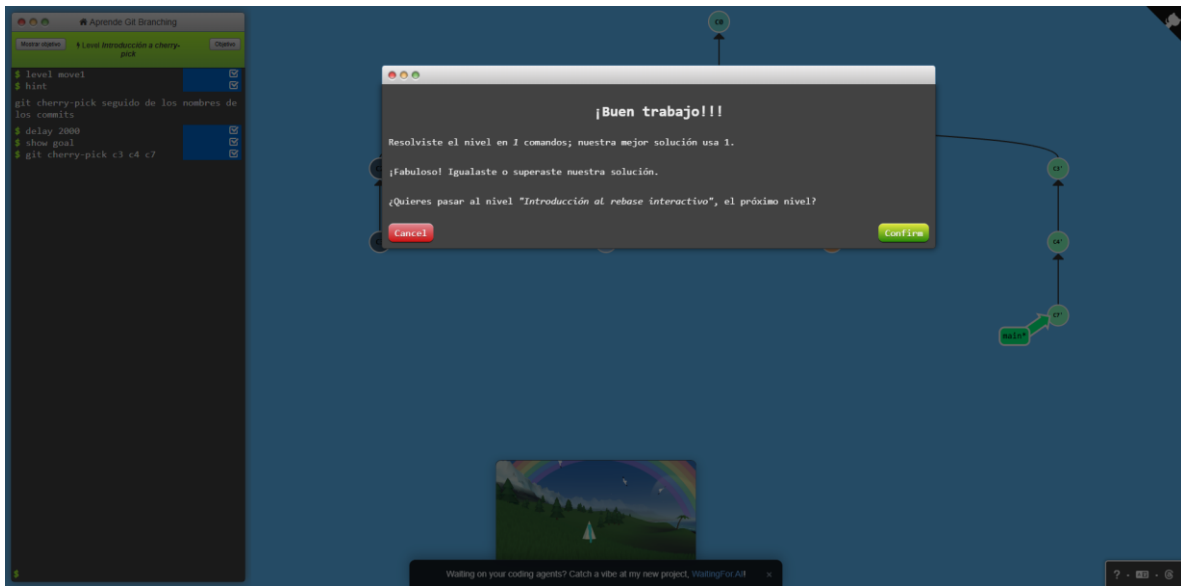
PUNTO 8:



Git Revert: Siempre va con un parámetro en la parte delantera para referenciarse de cual vamos a revisar para corregir y aparece C#* .

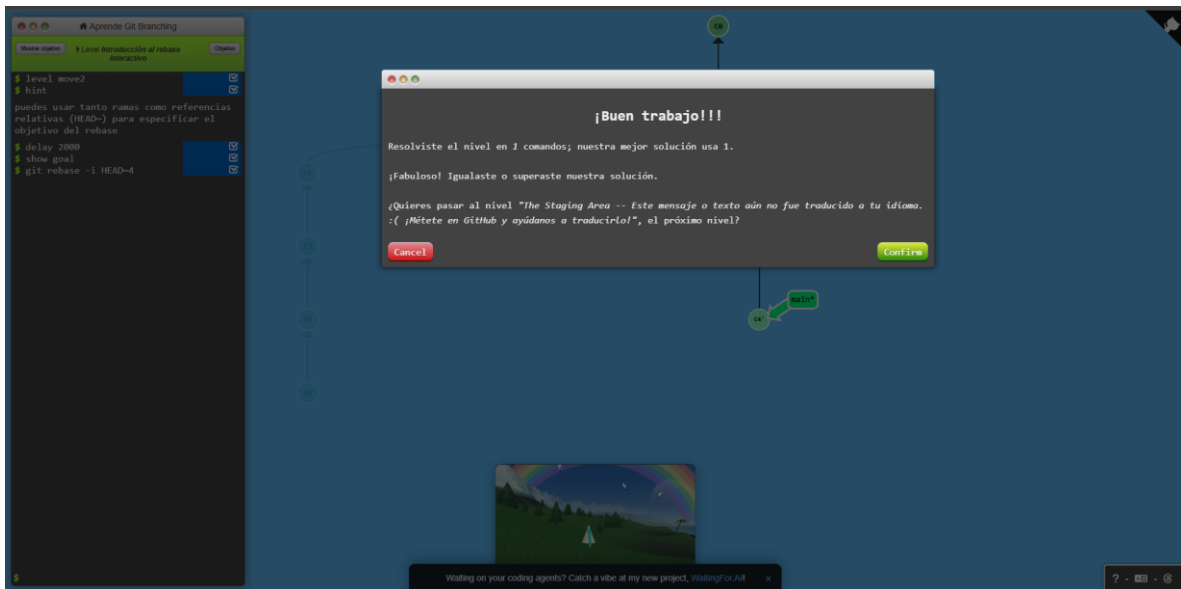
Git Reset: Lo mismo q el anterior de los parámetros pero a diferencia del anterior borra esa parte localmente.

PUNTO 9:



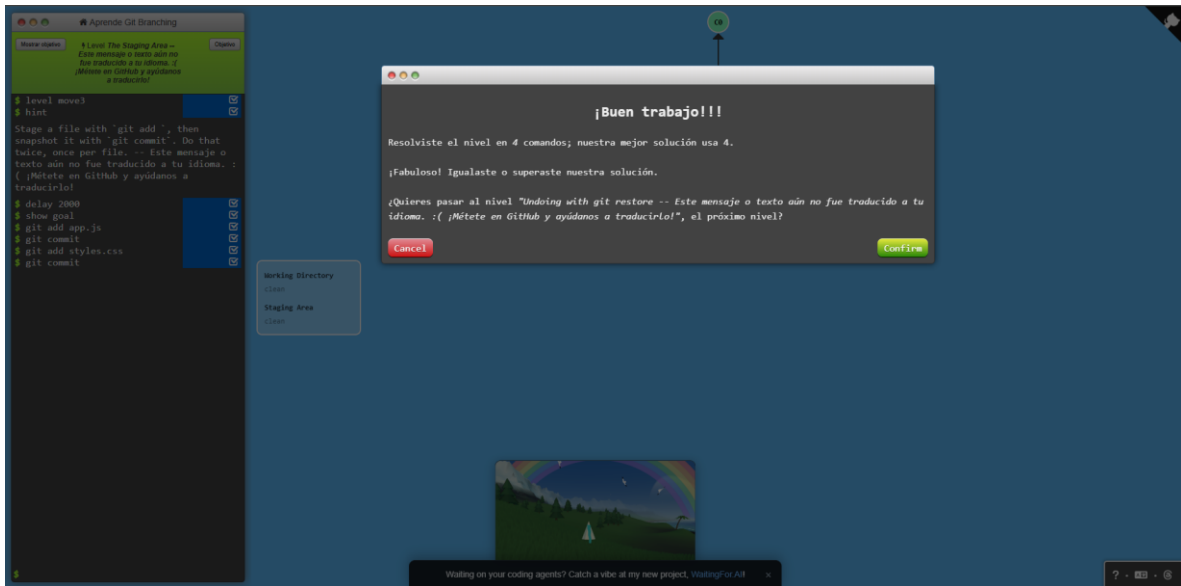
git cherry-pick: `git cherry-pick <Commit1> <Commit2> <...>`. Es una manera bastante directa de decir que quieres copiar una serie de commits sobre tu ubicación actual (HEAD).

PUNTO 10:



Git rebase -i HEAD~4: Permite seleccionar varios commits y organizarlos a gusto.

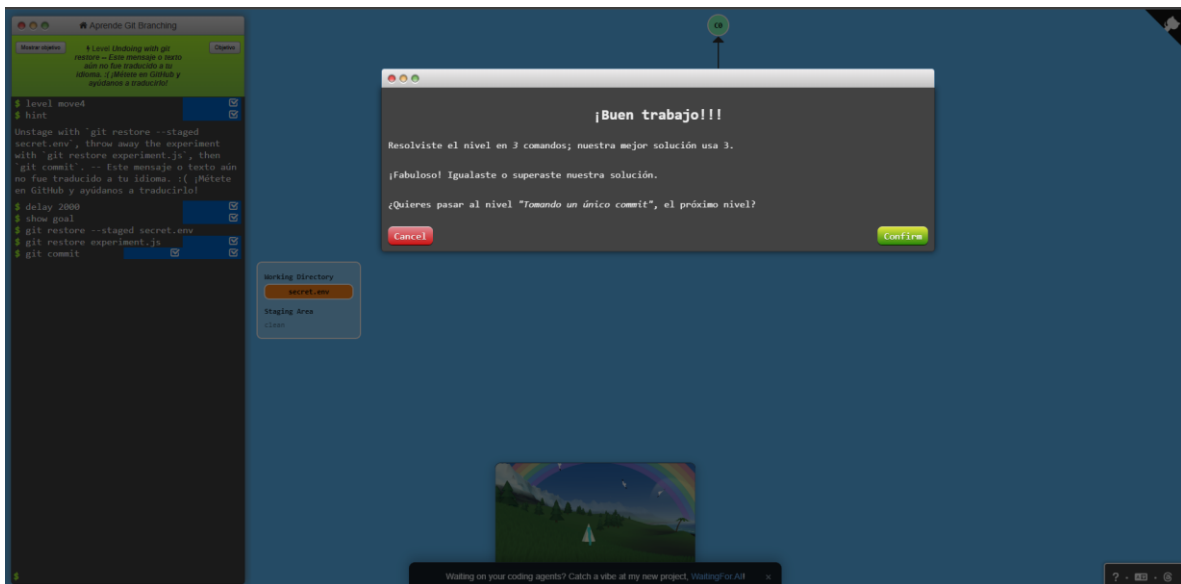
PUNTO 11:



- git add app.js, then git commit
- git add styles.css, then git commit

Git Add: agraga el archivo seleccionado después del comando y luego se agraga al commit al momento de crearlo

PUNTO 12:



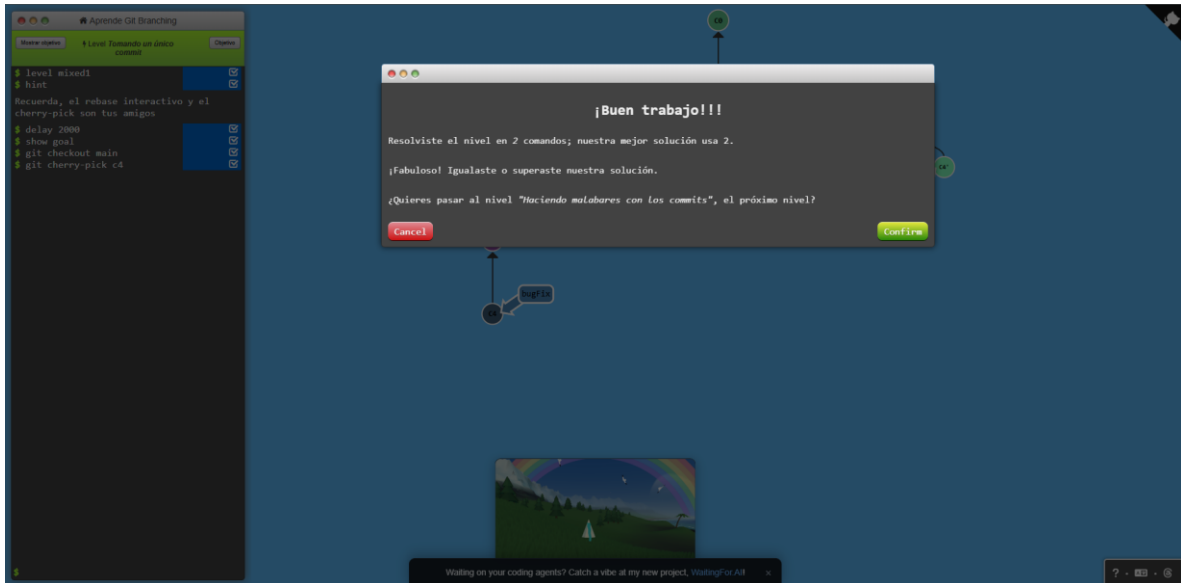
git restore --staged <file>: Saca un archivo del cual ya fue seleccionado mas no agregado por el momento

git restore <file>: Descarta por completo un archivo cargado

- Unstage the secret: git restore --staged secret.env

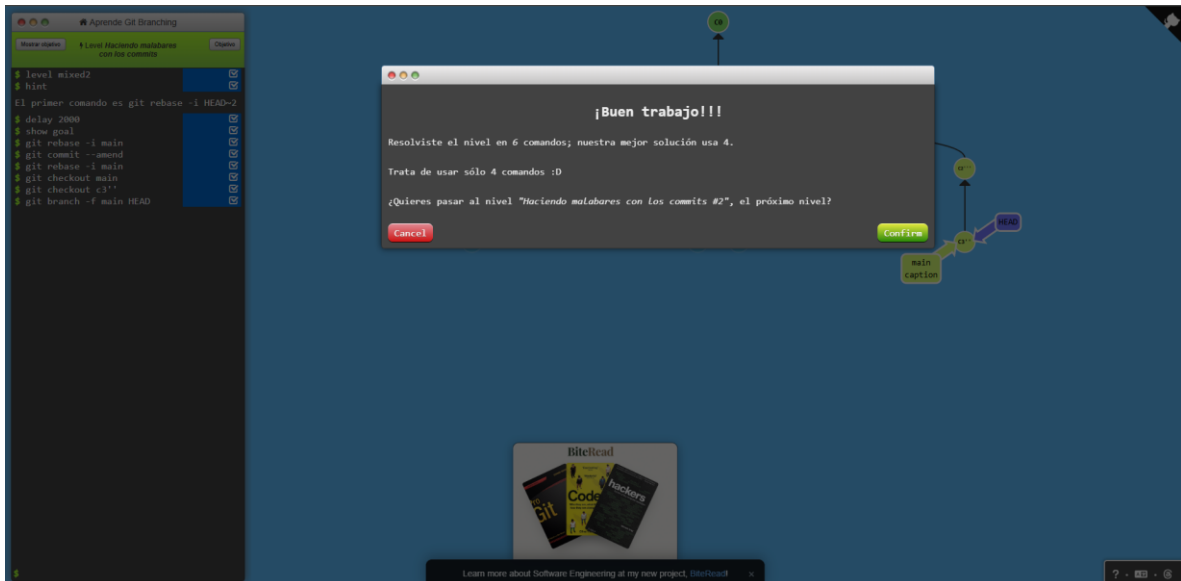
- Discard the experiment: git restore experiment.js
- Commit what's left: git commit

PUNTO 13:



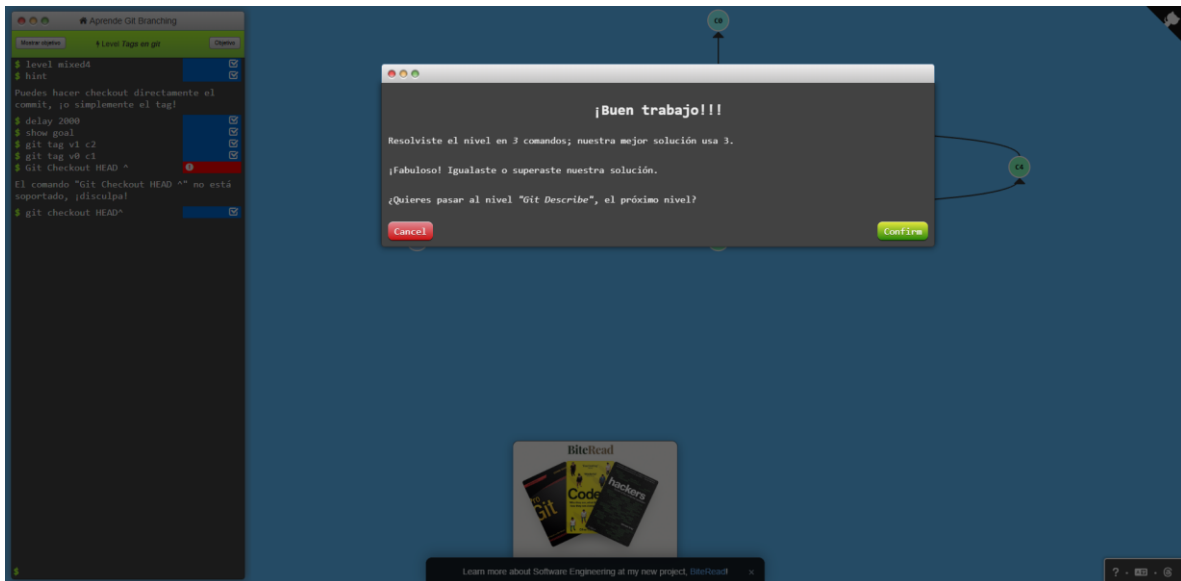
- git rebase -i
- git cherry-pick

PUNTO 14:



- Vamos a reordenar los commits para que el que queremos cambiar quede arriba de los demás con git rebase -i
- Vamos a hacer git commit --amend para aplicar la ligera modificación
- Después vamos a reordenar los commits a como estaban con git rebase -i
- Finalmente, vamos a mover main a esta parte actualizada de nuestro árbol de commits para terminar el nivel (usando el método que prefieras)

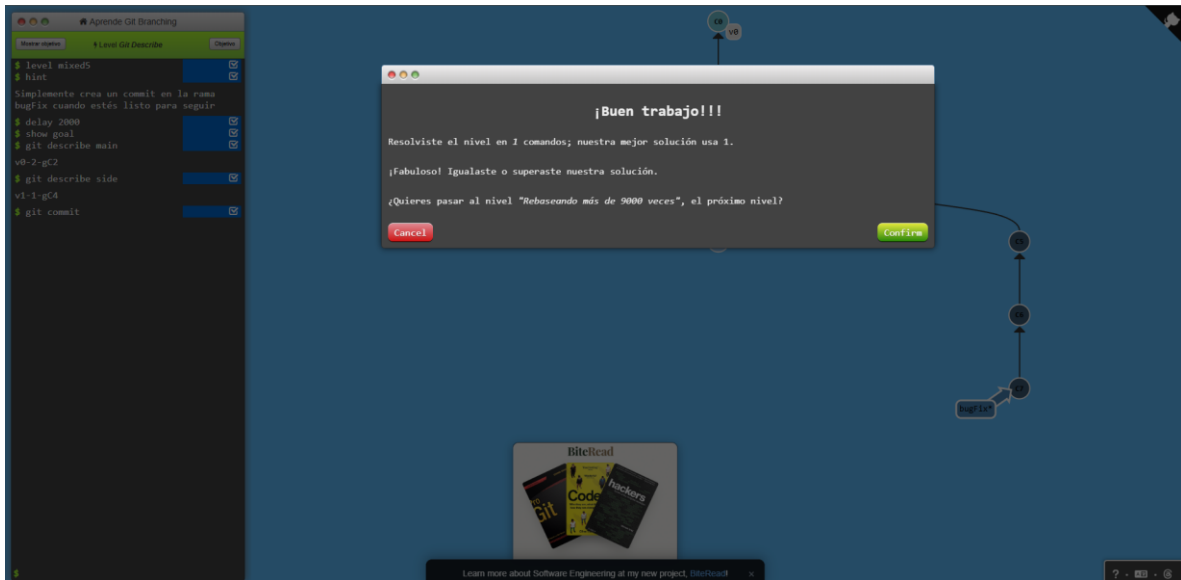
PUNTO 15:



Para este nivel, simplemente crea los tags en la visualización final y después haz checkout con v1. Observa cómo entras en el estado detached -- esto es porque no puedes hacer commit directamente sobre el tag v1.

Git tag <nombre> <donde va>: simplemente crea un tag en donde indiquemos necesitarlo

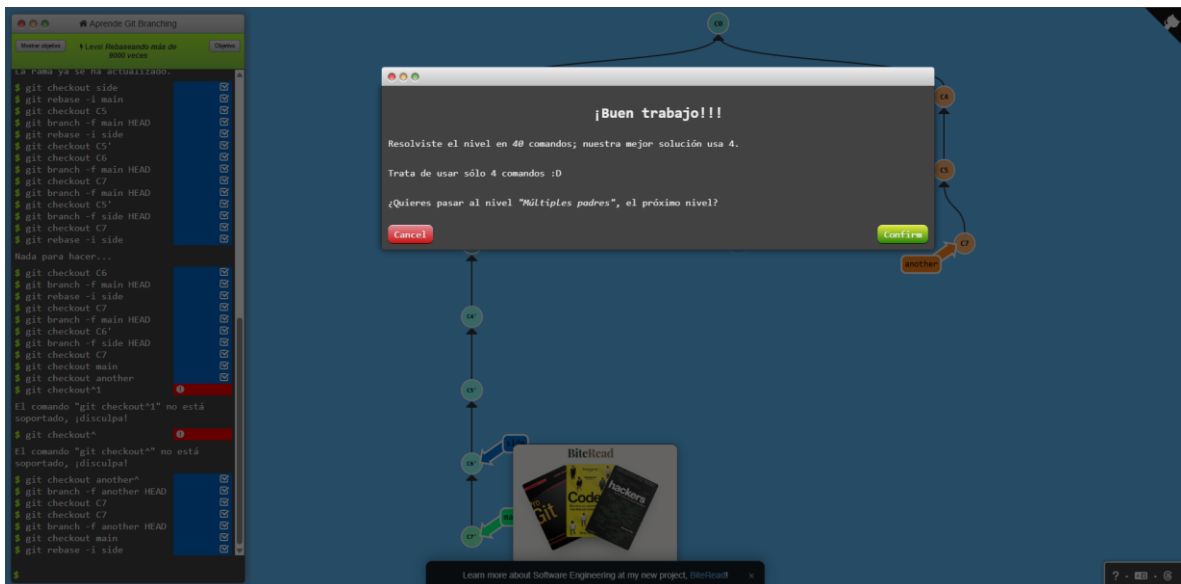
PUNTO 16:



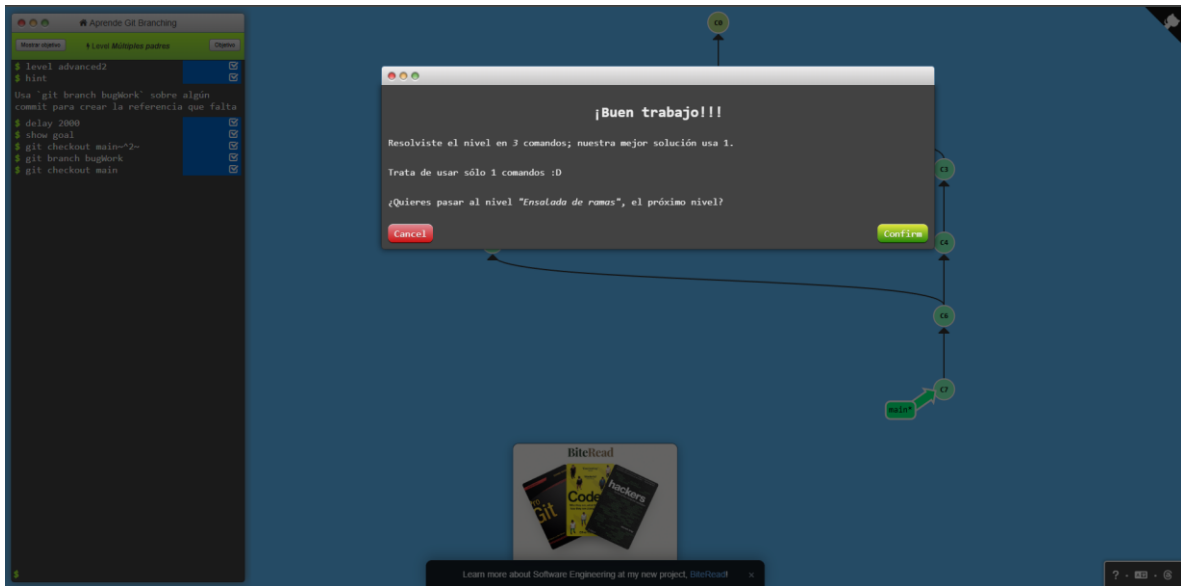
git describe <ref>: Donde <ref> es cualquier cosa que git puede resolver a un commit. Si no especificas ninguna referencia, git simplemente usa el commit en el que estás parado ahora (HEAD).

<tag>-<numCommits>-g<hash> : Donde tag es el tag más cercano en la historia, numCommits dice a cuántos commits de ese tag estás, y <hash> es el hash del commit que estás describiendo.

PUNTO 17:

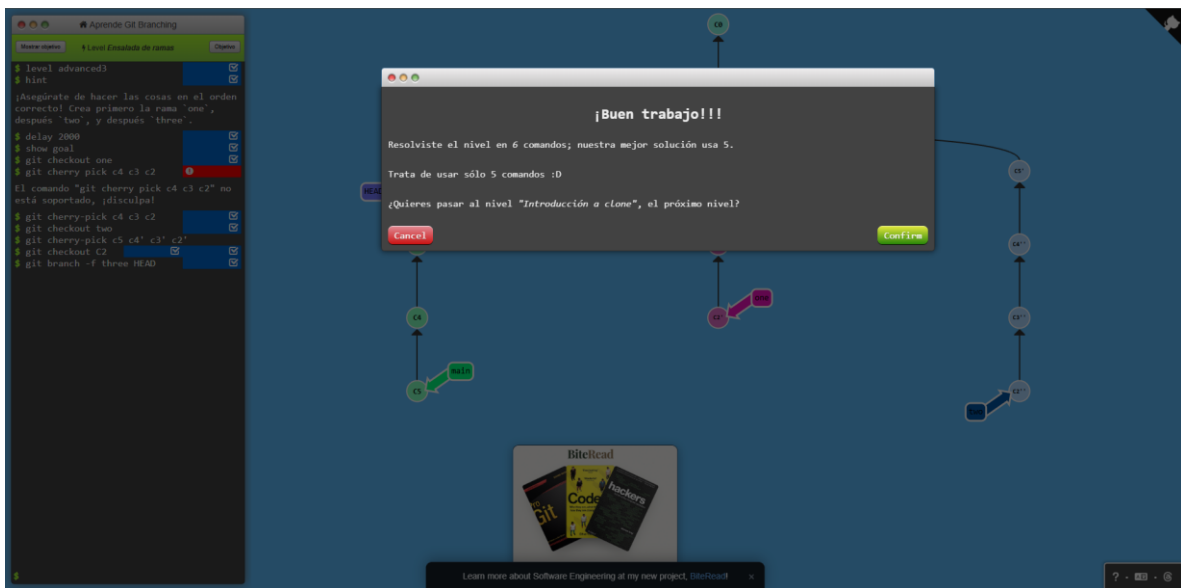


PUNTO 18:

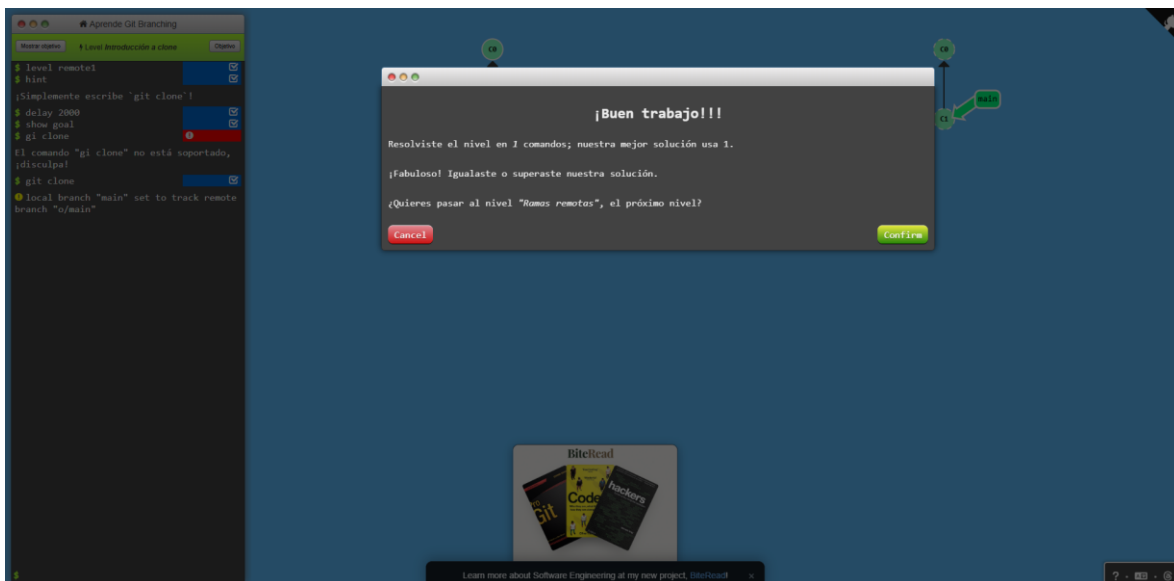


Git checkout (~) y (^): al momento de colocar (^) damos de cuenta un saldo de columna total en cambio (~) solo se devuelve un poco

PUNTO 19:



PUNTO 20:



GIT CLONE:

Clona todo el segmento seleccionado en un columna aparte